A. IDENTITAS

NIM : 2403024
Nama Lengkap : MUHAMMAD NURUL HUDA
Kelas : D3 TI2B
Program Studi : D3 TEKNIK INFORMATIKA
Jurusan : TEKNIK INFORMATIKA

B. DESKRIPSI MATA KULIAH

Nama Matakuliah : Pemrograman Perangkat Bergerak

C. RANGKUMAN TEORI

1. Class & Object
Class adalah cetakan, object adalah hasil dari class.
2. Property & Method
Property = data dalam class.
Method = fungsi/perilaku dalam class.
3. Constructor
Method khusus yang otomatis dipanggil saat objek dibuat.
4. Encapsulation
Menyembunyikan data agar tidak bisa diakses langsung dari luar class.
5. Inheritance
Pewarisan sifat dari class induk ke class anak.
6. Polymorphism
Satu method bisa memiliki banyak bentuk (perilaku berbeda di class turunan).
7. Abstraction
Menyembunyikan detail dan hanya menampilkan fungsi penting.
8. Interface
Kontrak yang harus diimplementasikan oleh class lain.
9. Mixin
Cara menggunakan fitur dari lebih dari satu class tanpa pewarisan langsung.
10. Static
Property atau method milik class, bukan milik objek.
11. this & super
this mengacu pada objek saat ini.
super mengacu pada parent class.
12. Late
Menunda inisialisasi variabel sampai digunakan.

D. Praktikum

1. Praktikum 1.1

```dart
class PrimaryButton {
  String? text;
  String? color;

  void render() {
    print("Merender tombol bertuliskan '$text' dengan warna $color.");
  }
}

Run | Debug
void main() {
  var loginBtn = PrimaryButton();
  loginBtn.text = "Login";
  loginBtn.color = "Biru";
  loginBtn.render();
}
```

```
PS D:\PPB\belajar_oop_dart\praktikum> dart praktikum_1_1.dart
Merender tombol bertuliskan 'Login' dengan warna Biru.
```

2. Praktikum 1.2

```
praktikum > praktikum_1_2.dart > main
```

```dart
class CounterWidget {
  final String title;
  CounterWidget({required this.title});
  _CounterState createState() => _CounterState();
}
class _CounterState {
  int _count = 0;
  int get count => _count;
  void increment() {
    _count++;
    print("Logika State: Data _count diubah menjadi $_count");
  }
  void build(CounterWidget widget) {
    print("Merender UI [${widget.title}] -> Teks angka di layar: $_count");
  }
}

Run | Debug
void main() {
  var myWidget = CounterWidget(title: "Penghitung Tiket");
  var state = myWidget.createState();

  state.build(myWidget);
  state.increment();
  state.build(myWidget);
}
```

```
PS D:\PPB\belajar_oop_dart\praktikum> dart praktikum_1_2.dart
Logika State: Data _count diubah menjadi 1
Merender UI [Penghitung Tiket] -> Teks angka di layar: 1
Memuat avatar dari 'profile_1.png' (ukuran: 40.0)
Apakah avatar1 identik dengan avatar2? true
```

3. Praktikum 2.1

```dart
class UserAvatar {
  final String imageUrl;
  final double size;

  const UserAvatar({required this.imageUrl, required this.size});

  void build() => print("Memuat avatar dari '$imageUrl' (ukuran: $size)");
}

Run | Debug
void main() {
  const avatar1 = UserAvatar(imageUrl: "profile_1.png", size: 40.0);
  const avatar2 = UserAvatar(imageUrl: "profile_1.png", size: 40.0);

  avatar1.build();
  print("Apakah avatar1 identik dengan avatar2? ${identical(avatar1, avatar2)}");
}
```

```
PS D:\PPB\belajar_oop_dart\praktikum> dart praktikum_2_1.dart
Memuat avatar dari 'profile_1.png' (ukuran: 40.0)
Apakah avatar1 identik dengan avatar2? true
```

4. Praktikum 2.2

```dart
class BaseScreen {
  final String routeName;
  BaseScreen(this.routeName);

  void navigate() => print("Beranjak ke layar: $routeName");
}
class ProfileScreen extends BaseScreen {
  final String username;
  ProfileScreen(String routeName, this.username) : super(routeName);

  void loadProfile() => print("Memuat data profil untuk $username...");
}

Run | Debug
void main() {
  var profile = ProfileScreen("/profile", "Huda aja");
  profile.navigate();
  profile.loadProfile();
}
```

```
PS D:\PPB\belajar_oop_dart\praktikum> dart praktikum_2_2.dart
Memuat data profil untuk Huda aja...
```

5. Praktikum 2.3

```dart
praktikum > praktikum_2_3.dart > PushNotification
class NotificationService {
  void send(String msg) => print("Mengirim notifikasi dasar: $msg");
}

class PushNotification extends NotificationService {
  @override
  void send(String msg) => print("[FCM Push Notif] Muncul di layar: $msg");
}

class EmailNotification extends NotificationService {
  @override
  void send(String msg) => print("[Email] Terkirim ke kotak masuk: $msg");
}

Run | Debug
void main() {
  NotificationService notif1 = PushNotification();
  NotificationService notif2 = EmailNotification();

  notif1.send("Ada pesan baru!");
  notif2.send("Laporan bulanan Anda.");
}
```

```
PS D:\PPB\belajar_oop_dart\praktikum> dart praktikum_2_3.dart
[Email] Terkirim ke kotak masuk: Laporan bulanan Anda.
```

6. Praktikum 2.4

```dart
abstract class BaseService {
  void logActivity(String message) {
    print("LOG SERVICE: $message");
  }
  void fetchApiData();
}
abstract class Syncable {
  void syncToLocal();
}
class UserDataService extends BaseService implements Syncable {
  @override
  void fetchApiData() {
    logActivity("Mengunggah endpoint Users...");
    print("Data pengguna berhasil diunduh.");
  }

  @override
  void syncToLocal() {
    print("Menyimpan data pengguna ke database SQLite lokal...");
  }
}

Run | Debug
void main() {
  var userService = UserDataService();
  userService.fetchApiData();
  userService.syncToLocal();
}
```

```
PS D:\PPB\belajar_oop_dart\praktikum> dart praktikum_2_4.dart
LOG SERVICE: Mengunggah endpoint Users...
Menyimpan data pengguna ke database SQLite lokal...
```

7. Praktikum 3.1

```dart
mixin FormValidator {
  bool isEmailValid(String email) => email.contains("@");
  bool isPasswordStrong(String pass) => pass.length >= 8;
}
class LoginController with FormValidator {
  void submitLogin(String email, String password) {
    if (isEmailValid(email) && isPasswordStrong(password)) {
      print("Validasi sukses! Memanggil API Login...");
    } else {
      print("Validasi gagal! Periksa format email/password.");
    }
  }
}
Run | Debug
void main() {
  var controller = LoginController();
  controller.submitLogin("admin.mail.com", "1234");
  controller.submitLogin("admin@mail.com", "passwordKuat");
}
```

```
PS D:\PPB\belajar_oop_dart\praktikum> dart praktikum_3_1.dart
Validasi gagal! Periksa format email/password.
Validasi sukses! Memanggil API Login...
Data dimuat. User ID U01 - flutter_ninja
Instance ApiClient dibuat (Hanya 1 kali seumur hidup aplikasi).
```

8. Praktikum 3.2

```dart
class UserModel {
  final String id;
  final String username;

  UserModel({required this.id, required this.username});
  factory UserModel.fromJson(Map<String, dynamic> json) {
    return UserModel(
      id: json['id'],
      username: json['username_api_field'],
    );
  }
}
class ApiClient {
  static final ApiClient _instance = ApiClient._internal();
  ApiClient._internal() {
    print("Instance ApiClient dibuat (Hanya 1 kali seumur hidup aplikasi).");
  }
  factory ApiClient() => _instance;
}

Run | Debug
void main() {
  Map<String, dynamic> responseAPI = {
    "id": "U01",
    "username_api_field": "flutter_ninja"
  };
  var user = UserModel.fromJson(responseAPI);
  print("Data dimuat. User ID ${user.id} - ${user.username}");
  var client1 = ApiClient();
  var client2 = ApiClient();
  print("Client1 & Client2 adalah objek yang sama? ${identical(client1, client2)}");
}
```

```
PS D:\PPB\belajar_oop_dart\praktikum> dart praktikum_3_2.dart
Data dimuat. User ID U01 - flutter_ninja
Instance ApiClient dibuat (Hanya 1 kali seumur hidup aplikasi).
Client1 & Client2 adalah objek yang sama? true
```

9. Praktikum 3.3

praktikum > praktikum_3_3.dart > main

```dart
class NetworkException implements Exception {
  final String message;
  NetworkException(this.message);
}
class UserRepository {
  void fetchUsers(bool hasInternetConnection) {
    try {
      print("Memulai request ke server...");
      if (!hasInternetConnection) {
        throw NetworkException("Koneksi internet terputus (Timeout).");
      }
      print("Berhasil mengunduh data user.");
    } catch (e) {
      print("Error tertangkap di Repository! Melempar ke UI...");
      rethrow;
    }
  }
}

Run | Debug
void main() {
  var repo = UserRepository();
  try {
    repo.fetchUsers(false);
  } on NetworkException catch (e) {
    print("UI: Tampilkan Alert -> ${e.message}");
  } catch (e) {
    print("UI: Error tidak dikenal -> $e");
  } finally {
    print("UI: Sembunyikan icon loading (CircularProgressIndicator).");
  }
}
```

```
PS D:\PPB\belajar_oop_dart\praktikum> dart praktikum_3_3.dart
Error tertangkap di Repository! Melempar ke UI...
UI: Tampilkan Alert -> Koneksi internet terputus (Timeout).
```

10. Praktikum 3.4

praktikum > utils > theme_colors.dart > ThemeColors

```dart
class ThemeColors {
  static const String primary = "Biru";
  static const String error = "Merah";
}
```

praktikum > main.dart > ...

```dart
import 'utils/theme_colors.dart';

Run | Debug
void main() {
  print("Warna utama tombol adalah: ${ThemeColors.primary}");
  print("Warna teks peringatan adalah: ${ThemeColors.error}");
}
```

```
PS D:\PPB\belajar_oop_dart\praktikum> dart main.dart
Warna utama tombol adalah: Biru
Warna teks peringatan adalah: Merah
```

E. LATIHAN

1. Latihan 1.1

```
latihan > latihan_1_1.dart > main
class CustomText {
  String content;
  double fontSize;

  CustomText(this.content, this.fontSize);
  void build() {
    print("Menampilkan teks '$content' dengan ukuran font $fontSize.");
  }
}

Run | Debug
void main() {
  var teksSaya = CustomText('Halo Flutter', 16.5);
  teksSaya.build();
}
```

```
PS D:\PPB\belajar_oop_dart\latihan> dart latihan_1_1.dart
Menampilkan teks 'Halo Flutter' dengan ukuran font 16.5.
```

2. Latihan 1.2

```
latihan > latihan_1_2.dart > main
class ProductCard {
  final String productName;
  final double price;

  const ProductCard({required this.productName, required this.price});
  void build() {
    print("Membangun Card: $productName (Rp $price)");
  }
}

Run | Debug
void main() {
  const card1 = ProductCard(productName: "Mouse Wireless", price: 150000.0);
  const card2 = ProductCard(productName: "Mouse Wireless", price: 150000.0);

  card1.build();
  bool isIdentical = identical(card1, card2);
  print("Card 1 dan Card 2 di memori yang sama? $isIdentical");
}
```

```
PS D:\PPB\belajar_oop_dart\latihan> dart latihan_1_2.dart
Membangun Card: Mouse Wireless (Rp 150000.0)
Card 1 dan Card 2 di memori yang sama? true
```

3. Latihan 2.1

```
latihan > latihan_2_1.dart > main
class SwitchWidget {
  _SwitchState createState() => _SwitchState();
}
class _SwitchState {
  bool _isOn = false;
  void build() {
    String status = _isOn ? "Menyala" : "Mati";
    print("Merender UI: Tombol Switch saat ini [$status]");
  }
  void toggle() {
    print("Logika State: Switch ditekan!");
    _isOn = !_isOn;
    build();
  }
}

Run | Debug
void main() {
  var mySwitch = SwitchWidget();
  var state = mySwitch.createState();
  state.build();
  state.toggle();
  state.toggle();
}
```

```
PS D:\PPB\belajar_oop_dart\latihan> dart latihan_2_1.dart
Merender UI: Tombol Switch saat ini [Mati]
Logika State: Switch ditekan!
Merender UI: Tombol Switch saat ini [Menyala]
Logika State: Switch ditekan!
Merender UI: Tombol Switch saat ini [Mati]
```

4. Latihan 2.2

latihan > latihan_2_2.dart > main

```dart
class BaseDialog {
  void show() {
    print("Menampilkan pop-up dialog di layar...");
  }
}
class ErrorDialog extends BaseDialog {
  String errorMessage;

  ErrorDialog(this.errorMessage);

  void logError() {
    print("Menyimpan log error: $errorMessage");
  }
}

Run | Debug
void main() {
  var errorDialog = ErrorDialog("Koneksi terputus!");
  errorDialog.show();
  errorDialog.logError();
}
```

```
PS D:\PPB\belajar_oop_dart\latihan> dart latihan_2_2.dart
Menampilkan pop-up dialog di layar...
Menyimpan log error: Koneksi terputus!
```

5. Latihan 2.3

```dart
class PaymentMethod {
  void processPayment() {
    print("Memproses pembayaran...");
  }
}
class EWallet extends PaymentMethod {
  @override
  void processPayment() {
    print("Memproses pembayaran menggunakan E-Wallet (QRIS)...");
  }
}
class BankTransfer extends PaymentMethod {
  @override
  void processPayment() {
    print("Memproses pembayaran menggunakan Virtual Account Bank...");
  }
}
Run | Debug
void main() {
  var eWallet = EWallet();
  var bankTransfer = BankTransfer();

  eWallet.processPayment();
  bankTransfer.processPayment();
}
```

```
PS D:\PPB\belajar_oop_dart\latihan> dart latihan_2_3.dart
Memproses pembayaran menggunakan E-Wallet (QRIS)...
Memproses pembayaran menggunakan Virtual Account Bank...
```

6. Latihan 2.4

```dart
abstract class BaseWidget {
  void showTooltip() {
    print("Menampilkan tooltip...");
  }
  void build();
}
abstract class Clickable {
  void onClick();
}
class CustomButton extends BaseWidget implements Clickable {
  @override
  void build() {
    print("Me-render CustomButton ke layar...");
  }

  @override
  void onClick() {
    print("Klik terdeteksi! Mengeksekusi aksi...");
  }
}

Run | Debug
void main() {
  var button = CustomButton();
  button.showTooltip();
  button.build();
  button.onClick();
}
```

```
PS D:\PPB\belajar_oop_dart\latihan> dart latihan_2_4.dart
Menampilkan tooltip...
Me-render CustomButton ke layar...
Klik terdeteksi! Mengeksekusi aksi...
```

7. Latihan 3.1

```dart
mixin CurrencyFormatter {
  String formatToIDR(int amount) {
    return "Rp $amount";
  }
}
class ProductDetailScreen with CurrencyFormatter {
  void showPrice() {
    int harga = 25000;
    print("Harga Produk: ${formatToIDR(harga)}");
  }
}
Run | Debug
void main() {
  var screen = ProductDetailScreen();
  screen.showPrice();
}
```

```
PS D:\PPB\belajar_oop_dart\latihan> dart latihan_3_1.dart
Harga Produk: Rp 25000
```

8. Latihan 3.2

latihan > latihan_3_2.dart > main

```dart
class AppConfig {
  static final AppConfig _instance = AppConfig._internal();

  AppConfig._internal() {
    print("Config diinisialisasi.");
  }

  factory AppConfig() => _instance;
}
class ArticleModel {
  final String title;
  final String content;

  ArticleModel({required this.title, required this.content});

  factory ArticleModel.fromJson(Map<String, dynamic> json) {
    return ArticleModel(
      title: json['title'] ?? '',
      content: json['content'] ?? '',
    );
  }
}

Run | Debug
void main() {
  var config1 = AppConfig();
  var config2 = AppConfig();
  print("Apakah config 1 dan 2 identik? ${identical(config1, config2)}");

  var jsonData = {
    "title": "Belajar Dart 3",
    "content": "Materi tentang OOP"
  };
  var article = ArticleModel.fromJson(jsonData);
  print("Judul Artikel: ${article.title}");
}
```

```
PS D:\PPB\belajar_oop_dart\latihan> dart latihan_3_2.dart
Config diinisialisasi.
Apakah config 1 dan 2 identik? true
Judul Artikel: Belajar Dart 3
```

9. Latihan 3.3

latihan > latihan_3_3.dart > saveProfile

```dart
class ValidationException implements Exception {
  final String message;
  ValidationException(this.message);
}
void saveProfile(String name) {
  if (name.isEmpty) {
    throw ValidationException("Nama tidak boleh kosong");
  }
}
Run | Debug
void main() {
  try {
    saveProfile("");
  } on ValidationException catch (e) {
    print("UI: Menampilkan pesan error -> ${e.message}");
  } finally {
    print("UI: Selesai memproses aksi simpan.");
  }
}
```

```
PS D:\PPB\belajar_oop_dart\latihan> dart latihan_3_3.dart
UI: Menampilkan pesan error -> Nama tidak boleh kosong
UI: Selesai memproses aksi simpan.
```

10. Latihan 3.4

latihan > product.dart > Product

```dart
class Product {
  final String name;

  Product(this.name);
}
```

latihan > main.dart > main

```dart
import 'product.dart';

Run | Debug
void main() {
  var myProduct = Product("Laptop Gaming");
  print("Class dari folder models berhasil diakses!");
}
```

```
PS D:\PPB\belajar_oop_dart\latihan> dart main.dart
Class dari folder models berhasil diakses!
```

F. TUGAS

```
PS D:\PPB\belajar_oop_dart\tugas> dart main.dart
=== E-Wallet System v1.0.0 ===

User: John Doe (john@example.com) - Saldo: Rp 100000.0

--- Deposit ---
Deposit berhasil: Rp 50000.0
Transaksi ditambahkan: Deposit - Rp 50000.0
User: John Doe (john@example.com) - Saldo: Rp 150000.0

--- Payment E-Wallet ---
Memproses pembayaran E-Wallet sebesar Rp 30000.0
Penarikan berhasil: Rp 30000.0
Transaksi ditambahkan: Pembayaran E-Wallet - Rp -30000.0
User: John Doe (john@example.com) - Saldo: Rp 120000.0

--- Payment Bank Transfer ---
Memproses pembayaran Bank Transfer sebesar Rp 20000.0
Penarikan berhasil: Rp 20000.0
Transaksi ditambahkan: Pembayaran Bank Transfer - Rp -20000.0
User: John Doe (john@example.com) - Saldo: Rp 100000.0

--- Transfer ---
Transfer Rp 25000.0 ke jane@example.com berhasil
Transaksi ditambahkan: Transfer ke Jane - Rp -25000.0
User: John Doe (john@example.com) - Saldo: Rp 75000.0

--- Test Insufficient Balance ---
Error: InsufficientBalanceException: Saldo tidak mencukupi. Saldo: Rp 75000.0

--- Test Invalid Amount ---
Error: InvalidAmountException: Jumlah deposit tidak valid

=== Riwayat Transaksi ===
[2026-03-02 14:51:18.173912] TRX1772437878171 - Deposit: Rp 50000.0
[2026-03-02 14:51:18.177716] TRX1772437878177 - Pembayaran E-Wallet: Rp -30000.0
[2026-03-02 14:51:18.181692] TRX1772437878181 - Pembayaran Bank Transfer: Rp -20000.0
[2026-03-02 14:51:18.183684] TRX1772437878183 - Transfer ke Jane: Rp -25000.0
==========

=== Program Selesai ===
```

G. REFLEKSI

Ketika pertama kali aku mencoba mempelajari OOP di Dart, perasaan aku campur aduk kaya semen di campur sama pasir. Di satu sisi, aku antusias karena ini adalah bagian penting di Flutter development.Kendalanya paling cuma error dan typo.